

List Abstract Data Type

James Anderson

CS 3100/5100

Wright State University

List ADT

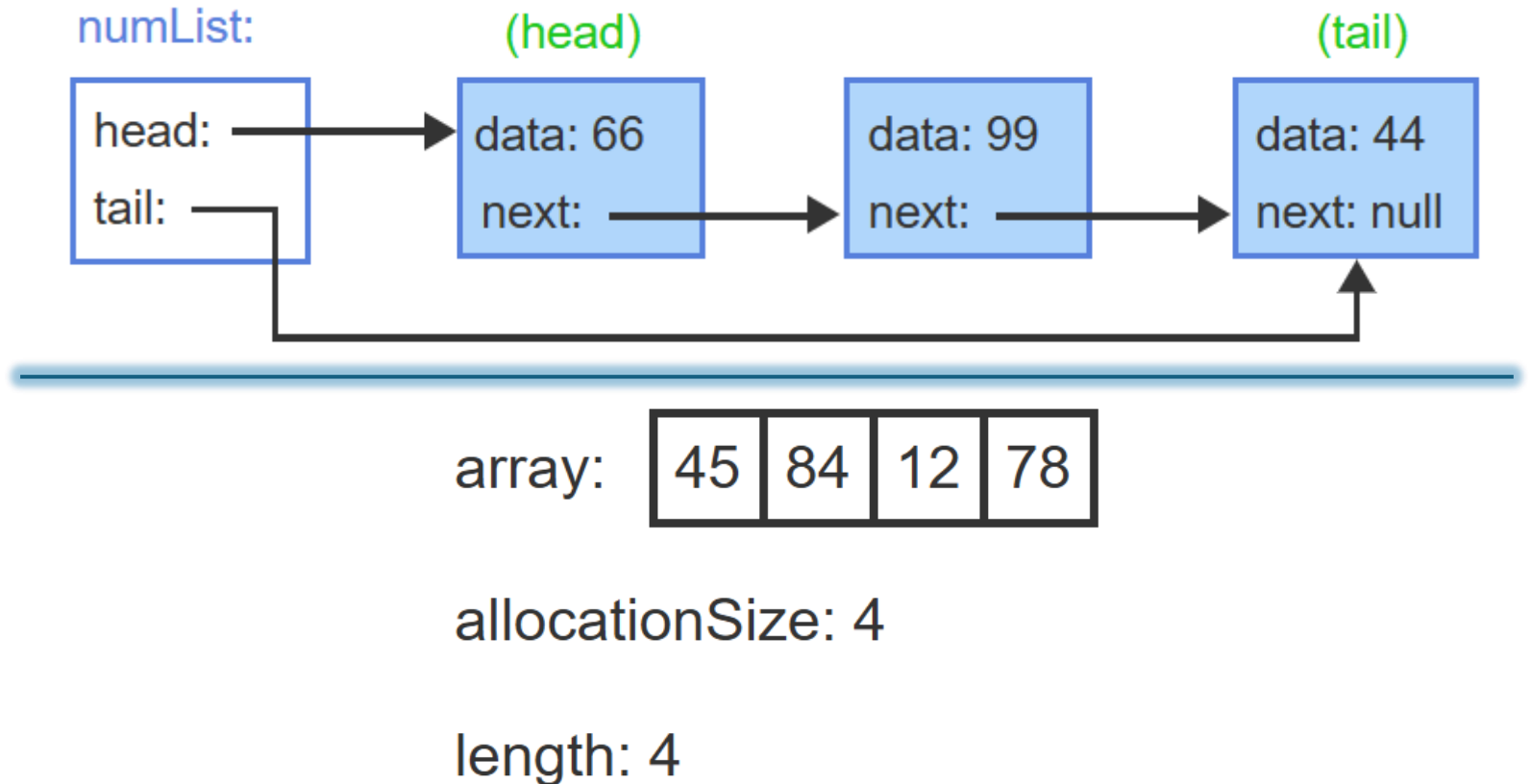
Operation	Description	Example starting with list: (99, 77)
Append(X)	Inserts X at end of list	<code>list.Append(44)</code> list: (99, 77, 44)
Prepend(X)	Inserts X at start of list	<code>list.Prepend(44)</code> list: (44, 99, 77)
InsertAfter(W, X)	Inserts X after W	<code>list.InsertAfter(99, 44)</code> list: (99, 44, 77)
Remove(X)	Removes X	<code>list.Remove(77)</code> list: (99)
Contains(X)	Returns true if X is in the list, false otherwise	<code>list.Contains(99)</code> returns true <code>list.Contains(22)</code> returns false
Print()	Prints list's items in order	<code>list.Print()</code> outputs: 99, 77
Sort()	Sorts list's items in ascending order	<code>list.Sort()</code> list: (77, 99)
IsEmpty()	Returns true if list has no items	<code>list.IsEmpty()</code> returns false
GetLength()	Returns the number of items in the list	<code>list.GetLength()</code> returns 2

Examples of some other List operations found in C++

front()	Return first item in list
back()	Return last item in list
at(i)	Return item at index
set(i, v)	Set the item at index <i>i</i> to the given value <i>v</i>
insert(v,i)	Insert element <i>v</i> at index <i>i</i> , values after <i>i</i> are moved 1 position up
popBack()	Remove last item in list
popFront()	Remove first item in list
clear()	Clears the contents, list size becomes zero

List Implementation

- Because List is an abstract data type (ADT), can be implemented a variety of ways:
- Linked-list
- Array



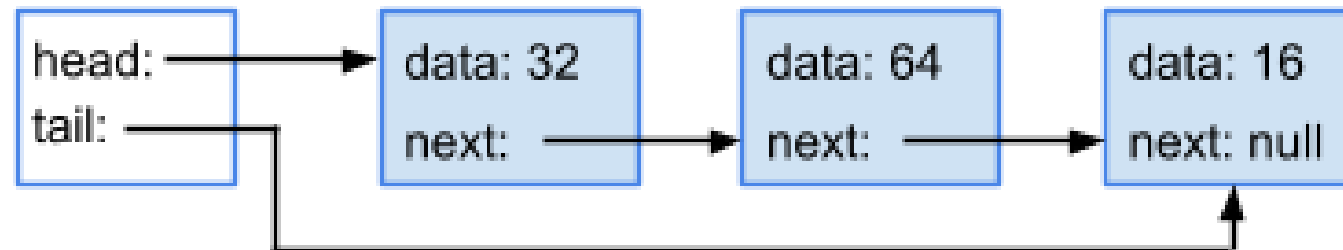
Singly-linked Lists

- Data structure with nodes
- List has two members: `head` and `tail`
- If list is empty, `head` and `tail` are null
- Positional – items are ordered

SinglyLinkedList

SinglyLinkedListNode* head

SinglyLinkedListNode* tail



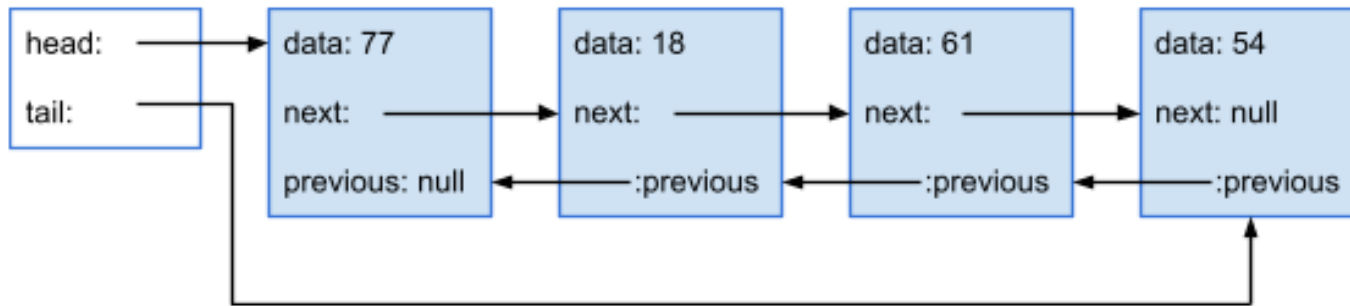
Singly-linked Nodes

- Stores a `value` and a pointer to the `next` node in the list
- If `next` is null, node is end of list

SinglyLinkedListNode
ValueType value
SinglyLinkedListNode* next

Doubly-linked Lists and Nodes

- List is the same with head and tail
- Nodes contain pointer to `next` and `previous` node in list
- Allows traversals forwards and backwards

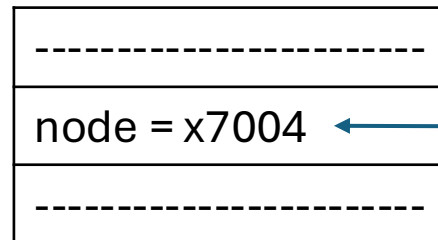
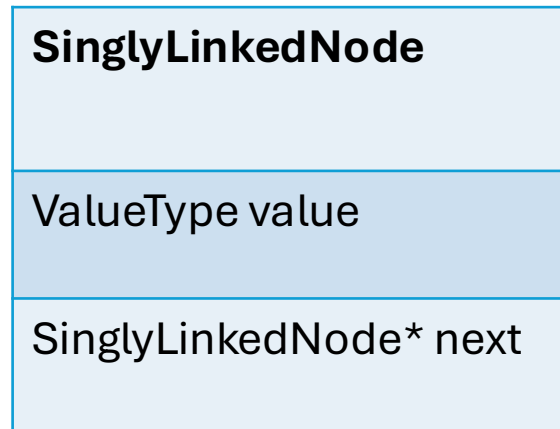


DoublyLinkedListNode
ValueType value
DoublyLinkedListNode* next
DoublyLinkedListNode* previous

Linked-list Node Creation in C++

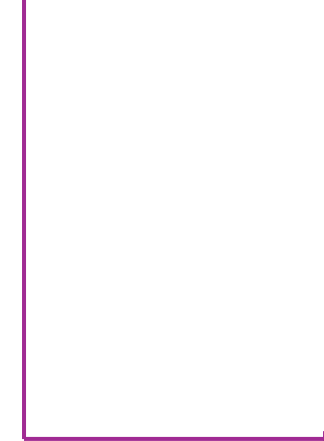
- Linked-list stores **pointers** to nodes
- `SinglyLinkedListNode*` syntax for a pointer to a node
- `new` keyword: allocate memory for a node, return the address of that memory
- Memory is allocated on the **heap**

```
SinglyLinkedListNode* node = new SinglyLinkedListNode() ;
```



heap memory

x7000	
x7001	
x7002	
x7003	
x7004	value
x7005	next
x7006	
x7007	
x7008	
x7009	
x7010	



Implementing List ADT with Linked-list

- Constructor for `SinglyLinkedList`: initialize `head` and `tail` to `nullptr`
- `nullptr` will mean there is no node
- C++ class: `ClassName::methodName()`

```
✓ SinglyLinkedList::SinglyLinkedList() {  
    head = nullptr;  
    tail = nullptr;  
}
```

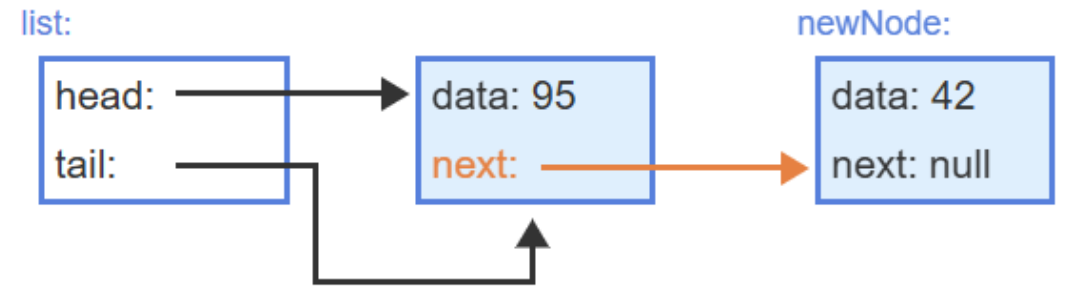
SinglyLinkedListNode class

- Constructor: pass node data
- Can only create new node if there is data
- Set data to `nodeData`
- Set `next` to `nullptr`

```
✓ SinglyLinkedListNode::SinglyLinkedListNode(const std::string &nodeData) {  
    ↘ data = nodeData;  
    next = nullptr;  
}
```

SinglyLinkedList: append

- Insert a newNode after the list's tail
- If list is empty: head and tail are nullptr, assign head and tail to new node
- If list is not empty:
 - Assign tail.next to new node
 - Assign tail to newNode
- Can break down into append(item) and appendNode(newNode)



append and appendNode

- append is public
- appendNode can be private

```
void SinglyLinkedList::append(string item) {  
    SinglyLinkedListNode* newNode = new SinglyLinkedListNode(item);  
    appendNode(newNode);  
}  
  
void SinglyLinkedList::appendNode(SinglyLinkedListNode *newNode) {  
    if (head == nullptr) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        tail->next = newNode;  
        tail = newNode;  
    }  
}
```

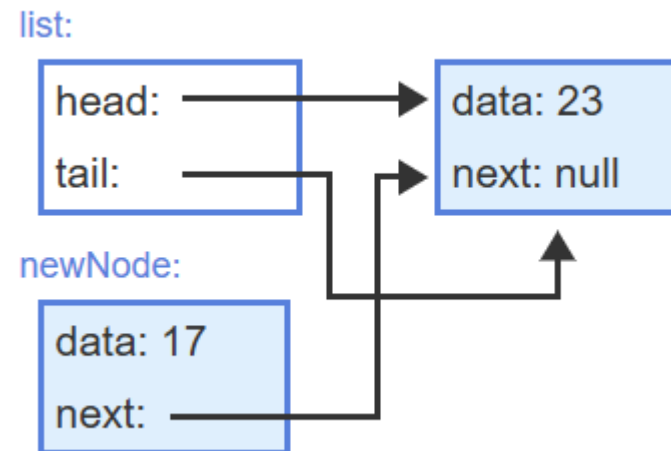
-> operator

- Values vs. Pointers
 - A value is the actual object
 - A pointer is an address to an object
- Values typically stored on runtime stack
- When dynamically allocating memory, object stored on heap
- Member of value can be accessed directly with object.member
- Pointer to object needs to be dereferenced with object->member

```
SinglyLinkedListNode nodeValue("value");  
SinglyLinkedListNode* nodePtr = new SinglyLinkedListNode("pointer");  
  
string nodeValueStr = nodeValue.data;  
string nodePtrStr = nodePtr->data;
```

SinglyLinkedList: prepend

- Insert a node in the front of the list, before the head
- If list is empty: head is null, assign head and tail with new node
- If list not empty: new node's next pointer assigned with head, head assigned with new node



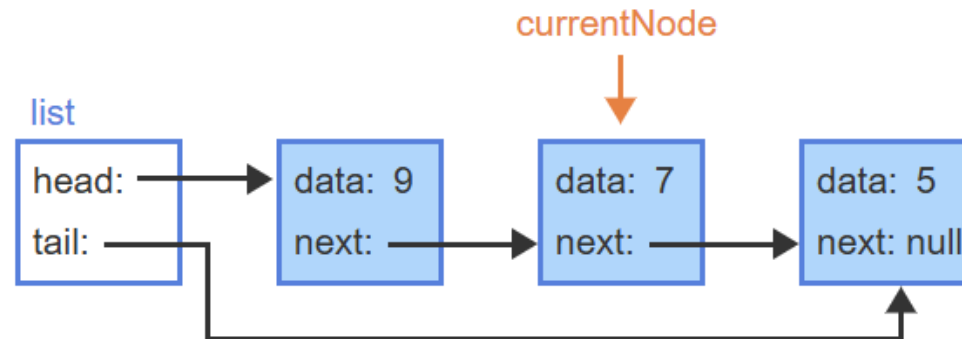
prepend and prependNode

```
void SinglyLinkedList::prepend(string item) {  
    SinglyLinkedListNode* newNode = new SinglyLinkedListNode(item);  
    prependNode(newNode);  
}  
  
void SinglyLinkedList::prependNode(SinglyLinkedListNode *newNode) {  
    if (head == nullptr) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        newNode->next = head;  
        head = newNode;  
    }  
}
```

Could also call empty()
Think OOP

SinglyLinkedList: search

- Return node pointer of node with search value if is in list, `nullptr` if not
- Start with `current` node assigned with `head`
- Check if `current` node's data matches search value,
 - If match, return node pointer
 - If mismatch, assign `current` to `current.next` and continue
- Once `current` is `nullptr`, return `nullptr`, reached end of list



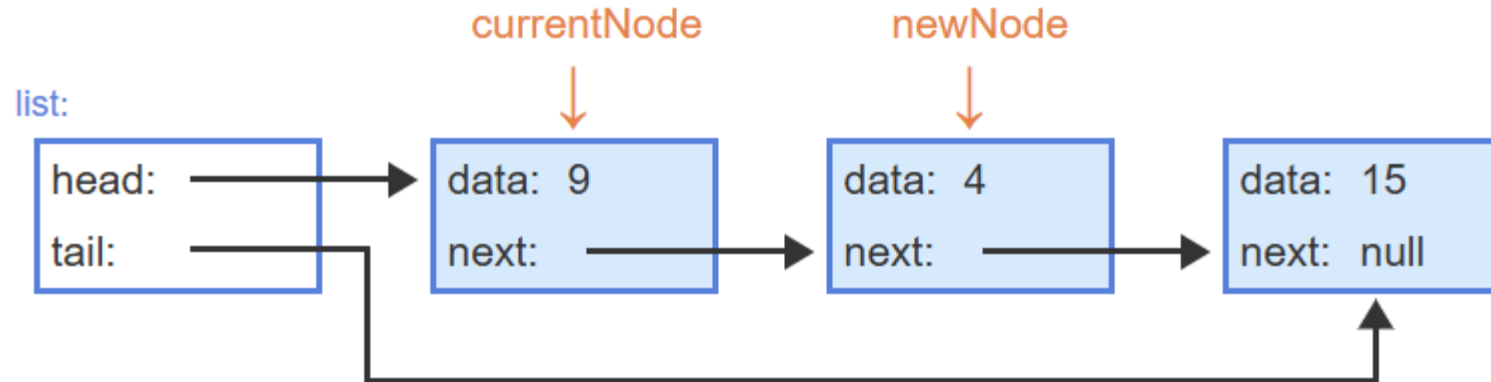
search

```
SinglyLinkedListNode* SinglyLinkedList::search(const string& item) const {  
    SinglyLinkedListNode* current = head;  
    while (current != nullptr) {  
        if (current->data == item) {  
            return current;  
        }  
    }  
    return nullptr;  
}
```

SinglyLinkedList: insert-node-after

- Given an existing node, insert `newNode` after
- `currentNode` is pointer to existing node, if list is empty `nullptr`
- Insert as list's first node: if `head` is `nullptr`, `head` and `tail` point to `newNode`
- Insert after `tail`: If list not empty and `currentNode` points to `tail` node, point `tail.next` to `newNode`, assign `tail` with `newNode`
- Insert in middle: if list not empty, `currentNode` does not point to `tail` node, then point `newNode.next` to `currentNode.next`, point `currentNode.next` to `newNode`

insertNodeAfter



```
void SinglyLinkedList::insertNodeAfter(SinglyLinkedListNode* currentNode, SinglyLinkedListNode* newNode) {  
    if (head == nullptr) {  
        head = newNode;  
        tail = newNode;  
    } else if (currentNode->next == nullptr) {  
        tail->next = newNode;  
        tail = newNode;  
    } else {  
        newNode->next = currentNode->next;  
        currentNode->next = newNode;  
    }  
}
```

SinglyLinkedList: insert-after

- List ADT has parameters for list *items*, not *nodes*
- `List::insertAfter(int currentItem, int newItem)`
- **Use** search to find node with `currentItem`
- **If found, create** `newNode` **for** `newItem`
- **Use** `insertNodeAfter` **to insert** `newNode`

insertAfter

```
bool SinglyLinkedList::insertAfter(const string& currentItem, const string& newItem) {  
    SinglyLinkedListNode* currentNode = search(currentItem);  
    if (currentNode != nullptr) {  
        SinglyLinkedListNode* newNode = new SinglyLinkedListNode(newItem);  
        insertNodeAfter(currentNode, newNode);  
        return true;  
    }  
    return false;  
}
```

SinglyLinkedList: nodeAt

- List ADT can provide access to elements at a given `index`
- `nodeAt` returns a pointer to the node at the given `index`
- If the `index` is invalid, such as off the end of the list, `nodeAt` returns `nullptr`
- `nodeAt` should not modify the list, so we label it as `const`
- `const` in C++ means the method does not modify the object
- `const` methods can only call other `const` methods
- Non-`const` methods can call both `const` and non-`const`

nodeAt

- `size_t` is C++ for unsigned since indexes shouldn't be negative
- Is there a more efficient way to do this? What could we add to our class?

```
SinglyLinkedListNode* SinglyLinkedList::nodeAt(size_t index) const {  
    SinglyLinkedListNode* current = head;  
    for (size_t i = 0; i < index; i++) {  
        current = current->next;  
        if (current == nullptr) {  
            return nullptr;  
        }  
    }  
    return current;  
}
```

SinglyLinkedList: at

- List ADT at doesn't return a node pointer, but the item at the index
- at can call nodeAt, and return the data in the node
- If nodeAt returns nullptr, we have options
 - Return a default value (empty string, int zero, int -1)
 - Throw an exception, user would need to catch
 - C++ has a type called optional<T>, if there is nothing to return, we can return nullopt, similar to nullptr

at

```
optional<string> SinglyLinkedList::at(size_t index) const {  
    SinglyLinkedListNode* result = nodeAt(index);  
    if (result != nullptr) {  
        return *result->data;  
    }  
    return *nullopt;  
}
```

SinglyLinkedList: set

- Modify the value element at a given index position in the list
- Can work similar to at () and use nodeAt ()
- Get the nodeAt given index, update node->data
- Could return boolean if operation was successful, or just return void

```
void SinglyLinkedList::set(size_t index, const string& item) {  
    SinglyLinkedListNode* toUpdate = nodeAt(index);  
    if (toUpdate != nullptr) {  
        toUpdate->data = item;  
    }  
}
```

SinglyLinkedList: remove-node-after

- **Given specified** `currentNode`, **remove the one after** => `currentNode.next`
- **If** `currentNode` **is** `nullptr`, **head is being removed** (no node before head)
 - **Assign** `nodeToRemove` **with** `head`
 - **Assign** `head` **with** `head.next`
- **Else, assign** `nodeToRemove` **with** `currentNode.next`
 - **Assign** `succeedingNode` **with** `nodeToRemove.next`
 - **Assign** `currentNode.next` **with** `succeedingNode`

removeNodeAfter

```
void SinglyLinkedList::removeNodeAfter(SinglyLinkedListNode* currentNode) {  
    if (currentNode == nullptr) {  
        // special case, remove the head  
        SinglyLinkedListNode* nodeToRemove = head;  
        head = head->next;  
        delete nodeToRemove; // release the memory that was pointed to by nodeToRemove  
  
        if (head == nullptr) {  
            tail = nullptr; // last item was removed  
        }  
    } else if (currentNode->next != nullptr) {  
        SinglyLinkedListNode* nodeToRemove = currentNode->next;  
        SinglyLinkedListNode* succeedingNode = nodeToRemove->next;  
        currentNode->next = succeedingNode;  
        delete nodeToRemove; // release the memory that was pointed to by nodeToRemove  
  
        if (succeedingNode == nullptr) {  
            tail = currentNode; // tail was removed, set tail back to currentNode  
        }  
    }  
}
```

SinglyLinkedList: remove-item

- Like insert-after, List ADT remove-item takes an *item* to remove, not a *node*
- Find node containing item, keep track of previous
- Call removeNodeAfter passing in previous

remove

```
bool SinglyLinkedList::remove(const string& itemToRemove) {  
    // Traverse to the node with the data equal to itemToRemove  
    // keep track of previous node  
    SinglyLinkedListNode* previous = nullptr;  
    SinglyLinkedListNode* current = head;  
    while (current != nullptr) {  
        if (current->data == itemToRemove) {  
            removeNodeAfter(previous);  
            return true;  
        }  
  
        // advance to the next node  
        previous = current;  
        current = current->next;  
    }  
    // traversed the entire list, element not found  
    return false;  
}
```

Linked List Traversal

- Go through each node, do something with each one
- E.g. print, increment, find min/max
- Start with `current` at head, loop until `current` is `nullptr`

LinkedList Print

```
void SinglyLinkedList::print() const {  
    SinglyLinkedListNode* current = head;  
    cout << "{";  
    while (current != nullptr) {  
        cout << current->data;  
        current = current->next;  
        if (current != nullptr) {  
            cout << ", ";  
        }  
    }  
    cout << "}" << endl;  
}
```

This will print the list like:

{first, second, third}

Array-based lists

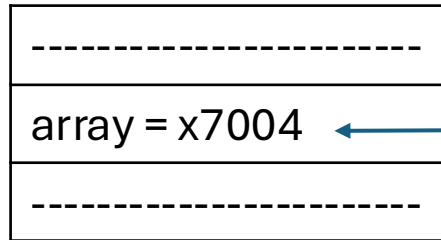
- Supports (mostly) the same operations as linked-list implementation
- C++ arrays: fixed size
- Initial array small
- When more space needed, reallocate array to larger size
- Typically 2x current size
- Keep track of:
 - allocationSize – size of array
 - length – number of elements in list

ArrayList
ValueType* data
int allocationSize
int length

Dynamic Array Allocation in C++

- Array list only stores a pointer to the first element
- Similar to when allocating for linked list nodes, use new keyword
- Since we want to store multiple items, we need to specify

```
int* array = new int[size];
```



heap memory

x7000	
x7001	
x7002	
x7003	
x7004	array[0]
x7005	array[1]
x7006	array[2]
x7007	array[3]
x7008	array[4]
x7009	
x7010	

size is variable, here it is 5

ArrayList::append

- Two conditions when append:
 - `length` is less than `allocationSize`
 - Since last valid index is at `length - 1`, the next is at `length`
 - Put item into `array[length]`, increment `length`
 - `length` is same as `allocationSize`
 - There is no more space in the array for a new element
 - `resize` array: create a new array with more space

append

```
void ArrayList::append(const string& item) {  
    if (length >= allocationSize) {  
        resize(allocationSize * 2);  
    }  
    data[length] = item;  
    length++;  
}
```

ArrayList::resize

- Typically increase by 2x
- Copy everything from old array into new array
- Release memory that old array was using
- `resize` does **not** impact the length of the list, only the amount of memory we have for the array
- `resize` can work to increase and decrease `allocatedSize`

resize

```
void ArrayList::resize(size_t newSize) {  
    string* newArray = new string[newSize];  
    allocationSize = newSize;  
    for (size_t i = 0; i < length; i++) {  
        newArray[i] = data[i];  
    }  
    delete[] data;  
    data = newArray;  
}
```

ArrayList::prepend and insertAt

- If we want to add to the front of the list, everything in the list needs pushed down
- This makes `prepend` $O(N)$
- `insertAt` has two cases:
 - Inserting somewhere in the middle $O(N)$
 - Inserting at the end (aka append) $O(1)$
- `prepend` can call `insertAt(item, 0)`

prepend and insertAt

```
void ArrayList::prepend(string item) {
    insertAt(item, index: 0);
}

void ArrayList::insertAt(const string& newItem, size_t index) {
    if (length >= allocationSize) {
        resize(allocationSize * 2);
    }
    for (int i = length; i > index; i--) {
        data[i] = data[i - 1];
    }
    data[index] = newItem;
    length++;
}
```

ArrayList::search

- Given an element, return the index of the first match
- If element not in list, we can:
 - Return -1
 - Use optional and return `nullopt`

search

```
optional<size_t> ArrayList::search(const string& index) const {  
    for (size_t i = 0; i < length; i++) {  
        if (data[i] == index) {  
            return i;  
        }  
    }  
    return nullopt;  
}
```

ArrayList::removeAt

- Given the index of an item, remove the item at that index.
- Each item after index moved down one position, length decreased
- Might need to remove first: $O(N)$

```
void ArrayList::removeAt(size_t index) {  
    if (index < length) {  
        for (size_t i = index; i < length - 1; i++) {  
            data[i] = data[i + 1];  
        }  
        length--;  
    }  
}
```